

Evidence-Governed Agent Harnesses

Making Evidence Survive Long-Running Agent Execution

What should an agent remember after a crash at step 40 of 60?

*Not just its state — but what it verified, what it accomplished,
and what it is still allowed to do.*

Khaja Shaik

Fifth Elephant Hyderabad | 2026



The production gap

Modern harnesses restore execution state. They do not necessarily restore decision-worthy evidence.

A checkpoint answers WHERE to resume. Safe continuation also needs WHAT was known, verified, still fresh, and authorized.

The distinction matters most in:

LONG-RUNNING

Evidence can expire before later actions.

CRASH + RESUME

Execution returns; evidence context may not.

CONSEQUENTIAL ACTIONS

Stale inputs can change the outcome.

A crash preserves state—not evidence

Implemented PoC · 10-step document analysis · evidence at steps 3, 6, 9 · crash at step 7

Steps 1–6 complete → step 7 crashes → recovery must decide what remains usable

What the trace can restore

Steps completed: 1–6
Outputs and resource records
Two evidence envelopes
Crash location: step 7
Execution can restart

What recovery must decide

Is step 3 evidence fresh?
Is step 6 evidence valid?
Was each envelope verified?
Must evidence be refreshed?
Is continuation authorized?

Evidence-governed recovery

Step 3 — Extract Entities	REFRESH	simulated stale evidence
Step 6 — Assess Risk	ACT	fresh and verified
Step 7 — Generate Findings	WAIT	resume only after required refresh

Decision: refresh stale evidence, then resume from step 7

Evidence is first-class runtime state

An envelope carries the decision context—not just the value

Source / Provenance	Where did it originate?
Observed at	When was it captured?
Freshness / Validity	Is it still applicable?
Verification	Was it checked, and by whom?
Content	What was observed?
Resource record	What did this step cost?
Outcome link	Which decision did it support?

A checkpoint stores position. An evidence envelope stores the basis for safe continuation.

The harness adds a policy gate to the loop

Checkpoint only where the next action depends on durable evidence

Traditional: Plan → Tool → Result → Next

EGAH: Plan → Checkpoint → Tool → Verify → Update evidence → Decide

At a consequential boundary

1. Capture execution + evidence state
2. Persist provenance, time and content
3. Verify against policy
4. ACT / REFRESH / ASK / ABSTAIN

After a crash

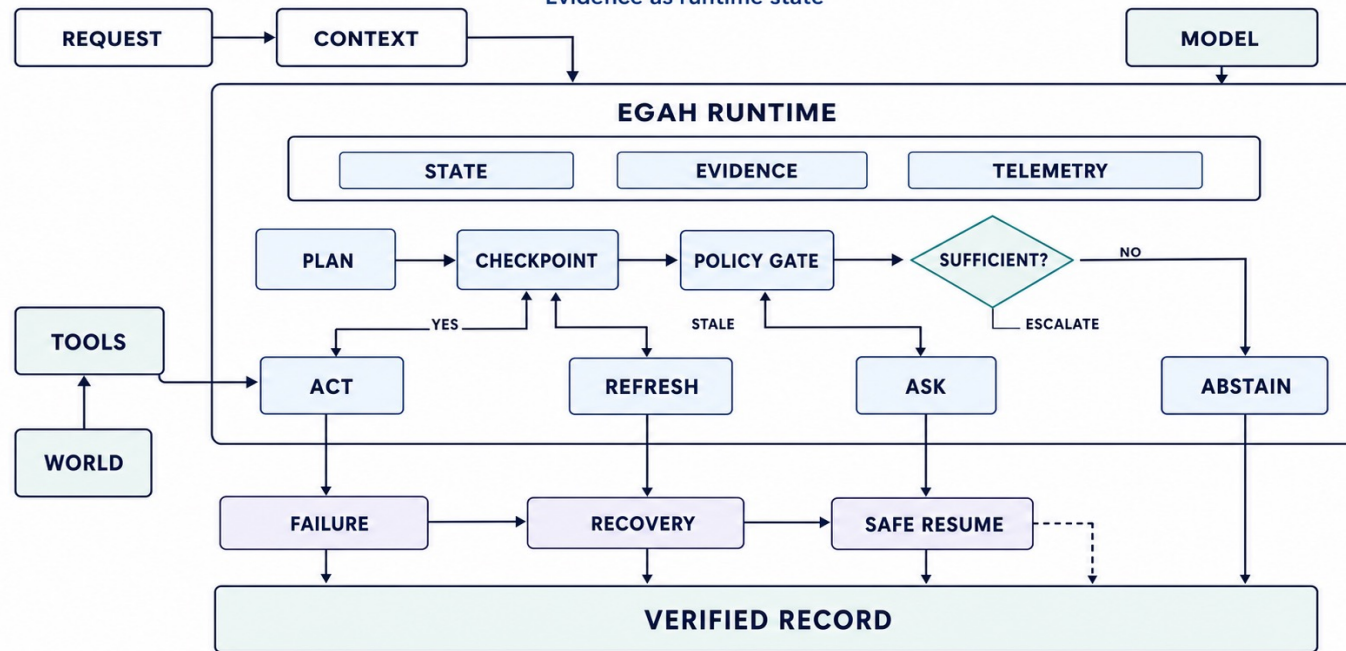
1. Restore state, envelopes and history
2. Recheck freshness and validity
3. Build a recovery plan
4. Resume, refresh, escalate—or stop

Architecture

End-to-end evidence-governed workflow

Evidence-Governed Agent Harness

Evidence as runtime state



Three core subsystems: State (graph, checkpoints, memory) · Evidence (provenance, validity, verification) · Telemetry (tokens, cost, latency)

Flow: Plan → Checkpoint → Policy Gate → Sufficient? → ACT / REFRESH / ASK / ABSTAIN

Recovery must restore the decision basis

Aspect	Traditional Recovery	EGAH Recovery
Execution state	Restored	Restored
Evidence state	Not checked	2 envelopes restored
Verification history	Not analysed	2 records analysed
Evidence freshness	Unknown	Step 3 stale; step 6 fresh
Resume decision	Resume at step 7	Refresh, then resume
Risk	May use stale evidence	Policy-gated continuation
Audit trail	Recovery gap	Continuous recovery record

Five-minute demo

Local Streamlit PoC · simulated mode works without an API key

- 1 Run the 10-step document workflow and inject a crash at step 7
- 2 Inspect two preserved evidence envelopes and verification records
- 3 Compare state-only recovery with evidence-governed recovery
- 4 Mark step 3 stale; generate a REFRESH recommendation
- 5 Resume safely and inspect the audit and resource trail
- 6 Local verification: 35 automated tests pass

The demo must show a changed recovery decision—not only a restored screen.

What should earn a place in the harness?

Six production checks

- 1 Separate execution state from evidence state
- 2 Persist provenance, freshness and verification
- 3 Revalidate at consequential boundaries
- 4 Use evidence-backed capability boundaries for smaller models
- 5 Link tokens and cost to verified outcomes
- 6 Measure reliability gain against harness overhead

After a crash, remember not only WHERE the agent was—but WHY it may continue.

Discussion: Which actions in your system deserve an evidence checkpoint?

Recovery has two separate failure modes

Execution recovery restores control flow; evidence-validity recovery restores permission to continue.

A resumed workflow is not safe merely because it is runnable. Required evidence must still be current, verified and authorized.

Two layers—one explicit boundary:

EXECUTION RECOVERY

Restores checkpoint,
state and next step.

EVIDENCE-VALIDITY RECOVERY

Rechecks freshness,
provenance and policy.

EGAH'S FOCUS

Turns validity into
ACT / REFRESH /
ASK / ABSTAIN.

EGAH complements the OSS stack

Capability	Existing OSS strength	Gap EGAH makes explicit
LangGraph	Graph checkpoints, replay, HIL	Evidence schema + validity gate
Temporal	Durable history and replay	External-fact freshness
Dapr Workflow	Retries; signed history	Decision-evidence lifecycle
Prefect	Flow/task state and retries	Resume-authorization policy
OpenTelemetry	Traces, metrics, logs, baggage	Executable continuation decision
Langfuse / Phoenix	LLM traces, evals, cost, latency	Durable evidence dependencies
EGAH	Composes above any runtime	ACT · REFRESH · ASK · ABSTAIN

Adapter-first integration

EGAH can sit beside existing workflow, data, policy and observability investments.

Current PoC: Python package + SQLite + Streamlit. Production interfaces below are a staged roadmap—not implemented claims.

Three community entry points:

EMBEDDED SDK

Python library around consequential nodes; store + decide.

RUNTIME HOOKS

LangGraph, Temporal, Dapr or Prefect; SQL/NoSQL evidence stores.

SERVICE + EVENTS

REST/gRPC; OTel export; OPA/Cedar; bureau, KYC, fraud and pricing adapters.

Safety gain has a measurable cost

Metric	State-only baseline	EGAH · 3 checkpoints
Median · 10 steps	13.53 ms	17.31 ms (+28.0%)
p95 · 10 steps	15.63 ms	21.24 ms (+35.8%)
Throughput	73.9 workflows/s	57.8/s (−21.9%)
Logical payload	6.70 KB / run	10.66 KB (+3.95 KB)
Recovery assessment	0.157 ms	0.539 ms (+0.382)
Stale-case block	0 / 200	200 / 200 unsafe resumes
Automated tests	Same codebase	35 / 35 passing

Operations move to targeted action

Event	State-only operating model	EGAH-enabled model
Failure	Restart at checkpoint	Assess evidence before restart
Alert	Crash, retry, latency	+ stale / invalid / policy signal
Incident triage	Correlate logs + DB manually	Run → evidence → verification
Support ticket	Broad rerun or manual checks	Refresh affected provider only
Audit query	Assemble proof after event	Query envelope + decision
Escalation	Operator infers next action	ASK / ABSTAIN with reason
Pilot KPI	MTTI + touch time unmeasured	Measure before / after; no claim

Loan journey: each decision earns a gate

Scenario demo · customer reaches signature after a long interruption

- 1 Application and consent captured
- 2 KYC and sanctions checks verified → evidence A
- 3 Bureau score 742; valid for 2 hours → checkpoint B
- 4 Fraud signals + underwriting policy evaluated → checkpoint C
- 5 Offer generated: 10.25%, clauses v17 → document hash D
- 6 Customer views offer; network fails before signature

Execution can resume at signature. The decision evidence supporting that agreement may no longer be valid.

Recovery must re-earn permission to sign

Three hours later · bureau expired · fraud signal changed · pricing moved

- 1 Restore “await signature”; load evidence A / B / C / D
- 2 Reassess: KYC current; bureau expired; fraud changed
- 3 Policy returns REFRESH; invalidate the v17 agreement
- 4 Re-fetch bureau + fraud; rerun affected underwriting
- 5 Pull current rate + clauses; generate v18 and a new hash
- 6 Expose only v18 for signature; append verification + audit

Safe continuation means the obsolete 10.25% agreement cannot be signed. Only the regenerated, currently justified offer remains active.

Implementation lessons shape the roadmap

Learning	What the PoC exposed	Next engineering step
Evidence semantics	A value alone is insufficient	Typed provider schemas + hashes
Freshness	TTL is only one invalidator	Events, versions, authorization
Checkpoint placement	Every step is wasteful	Risk-based action boundaries
Dependency refresh	Stale input invalidates outputs	Evidence dependency DAG
Atomicity	Separate commits can diverge	Transactional outbox + HA store
Enforcement	UI guard is not sufficient	Plan-bound backend resume API
Privacy	Evidence may be sensitive	Minimize, encrypt, retain, control

Where EGAH does not fit

Limitation	Current PoC truth	Why it matters
Verification	Output length > 10 chars	Not factual correctness
Staleness	Demo aging by step position	No live invalidation feed
Refresh	Recommendation only	No bureau / fraud adapter
Execution	Sequential Python loop	LangGraph graph not active
Persistence	SQLite; separate commits	Not HA or crash-atomic
Performance	+28% median local latency	Checkpoint selectively
Poor fit	Short, low-risk, deterministic	Policy/storage may not pay back

What this submission proves

Objective evidence for a practitioner-focused CFP

- 1 Problem is precise: resume position $\neq$ valid decision basis
- 2 Distinct gap: complements durable engines and observability
- 3 Working core: envelopes, verification, recovery plan, audit
- 4 Repeatable proof: 35/35 tests; 200/200 stale cases blocked
- 5 Measured cost: +28% latency; +3.95 KB logical payload/run
- 6 Honest boundary: live refresh, policy/authorization + HA remain

Evaluate EGAH as a composable evidence-governance layer at consequential recovery boundaries.

Reviewer decision: Is the control problem clear, testable and useful to practitioners?